

Ripple: An Open, AI-Formalized Lean 4 Framework for Computing with CRNs

Ho-Lin Chen^{*1} and Xiang Huang^{†2}

¹Department of Electrical Engineering, National Taiwan University, Taiwan,
holinchen@ntu.edu.tw ORCID 0000-0002-6171-9962

²Department of Computer Science, University of Illinois Springfield, USA, xhuan5@uis.edu
ORCID 0000-0002-4815-6130

Abstract

We present **Ripple**, an open, AI-formalized Lean 4 framework for the mathematics of computing real numbers with chemical reaction networks (CRNs). Ripple formalizes the full ladder of models — the GPAC / CRN continuum and the CRN-computable reals, the large-population-protocol (LPP) compilation pipeline, and a continuous-time Markov chain (CTMC) layer bridged to the deterministic mean-field limit by three machine-checked versions of Kurtz’s theorem. The development is reliable (its core constructions are verified to depend on exactly the three Mathlib foundational axioms, with no `sorry`); it exposed genuine, fixable gaps in published proofs (the approximate-majority convergence argument and the LPP main theorem); and it proves new results — a fully machine-checked construction of Apéry’s constant $\zeta(3)$ as a CRN-computable number via its holonomic generating function, the same recipe turning the modular $1/\pi$ series of Ramanujan into a sharp open problem. The formalization was carried out predominantly by AI agents using only publicly available models, so the workflow is reproducible.

Keywords: formal verification, Lean 4, chemical reaction networks, analog computation, GPAC, computable real numbers, population protocols, Kurtz’s mean-field theorem.

1 Introduction

Interactive theorem provers have lately moved from a curiosity at the edge of mathematics to a working tool inside it. Lean 4 and its library Mathlib now carry machine-checked proofs once thought out of reach — the Liquid Tensor Experiment, the polynomial Freiman–Ruzsa conjecture, an in-progress Fermat’s Last Theorem — and AI agents have begun proving theorems autonomously, settling open Erdős and OEIS problems [19] and disproving the Erdős unit-distance conjecture [16]. Those breakthroughs used proprietary internal models; the work here was done entirely with *publicly available* ones (Claude Opus 4.6–4.7, GPT-5.4–5.5), so the AI-assisted workflow is reproducible by anyone.

We bring this to molecular programming, with two concrete targets: the theory of which real numbers a chemical reaction network can compute, and the population-protocol *majority*

^{*}Supported in part by NSTC (Taiwan) grant 113-2221-E-002-204-MY3.

[†]Supported in part by Department of Energy EXPRESS grant DE-SC0024278.

algorithms. Both are exact, layered, and spread across many papers — the kind of mathematics where formalization earns its place by catching gaps that informal proofs leave behind. **Ripple** (<https://github.com/zinan-huang/Ripple>) is our Lean 4 formalization of both: an *open, AI-formalized* framework for computing with CRNs, built so that its foundations are reusable and its constructions can be extended.

The formalized theory. As an initial thread Ripple formalizes the theory of *CRN-computable real numbers*, building on three prior papers [12, 11, 7] and covering the full ladder of models:

- **Continuous / deterministic.** The GPAC / polynomial-IVP form, CRN mass-action dynamics, the CRN-computable reals, dual-rail compilation, and the large-population-protocol (LPP) refinement, under one shared Lean definition of “CRN computes α ”.
- **Stochastic / discrete.** The Angluin–Aspnes–Eisenstat 3-state *approximate majority* protocol with its full $O(n \log n)$ convergence proof [1], and the deterministic correctness chain of the Doty et al. optimal *exact majority* protocol [9].
- **The bridge.** Kurtz’s *mean-field limit theorem* [14], making precise how the deterministic ODE is the $N \rightarrow \infty$ limit of the stochastic CRN.

Four things formalization buys. Formalizing this theory does four things informal mathematics cannot.

1. **It is reliable.** A machine-checked proof leaves no step for a reader to fill in: every inference is checked by Lean’s small trusted kernel, so trust reduces to that kernel plus three foundational axioms. We verified, via Lean’s axiom-dependency check, that the core constructions ($\pi/4$, $\zeta(3)$, Catalan, LPP, Kurtz) depend on exactly those and nothing more — no **sorry**, no hidden hypotheses (§8).
2. **It exposes gaps.** The formalization uncovered genuine, fixable gaps in published constructions — in the original approximate-majority convergence proof and in the LPP main theorem among them — each caught as a Lean compile error or explicit counterexample (§4).
3. **It is open and extensible.** Adding a computable number is a plug-in; more broadly the foundation — the GPAC core, the CTMC/Kurtz layer, the compilation pipelines — is a base others can build new threads on (§2).
4. **It proves new things — and finds new problems.** Ripple gives a fully machine-checked construction of $\zeta(3)$ as a CRN-computable number via its holonomic generating function (§5); the same recipe applied to the modular $1/\pi$ series of Ramanujan surfaces a sharp open problem, its series anchor being a fixed point of the drive (§6).

Organization. §2 is an overview of *what the Lean development contains* — a map of the framework and the two threads it carries. §3 then develops the model ladder in full, from the GPAC / CRN continuum, through the foundational constants and the LPP compilation pipeline, to the CTMC layer, Kurtz’s theorem, and the LPP→Kurtz two-limit interchange. §4 records the gaps the formalization exposed; §5 and §6 then give the new $\zeta(3)$ and π constructions in detail, with both the informal arguments and the shape of the Lean proofs. Finally §7 reports how the formalization was actually done, and §8 states the trust footprint honestly.

2 An Open Framework: Foundations, Two Threads, and Extension

Ripple is built to be *open*: a reusable foundation that others can extend, or borrow from to build their own developments. It is best read not as a closed result but as an *initiative* — an initial attempt that carries two threads of research forward, on infrastructure deliberately made general. Two layers make this concrete. Figure 1 is a map of the repository — each pillar with its size and what it formalizes — showing both the reusable foundation and the threads (completed and in progress) built on it.

Ripple/ pillar	files / lines / defs	what it formalizes
Core/	13 / 6.7k / 136	GPAC/PIVP, bounded-time complexity, CRN pipeline
ODE/	1 / 0.3k / 4	Lyapunov / scalar convergence barriers
DualRail/	18 / 14k / 356	dual-rail encoding of polynomial dynamics
LPP/	25 / 80k / 1790	large-population protocols [11]; algebraic construction; balancing-dilation fix
Number/	21 / 14k / 565	$e, \pi, \ln 2, \gamma$, Catalan G , $\zeta(3)$ (Fermi/Apéry)
Number/Frobenius/	23 / 57k / 2068	Frobenius regular-singular theory; Apéry conifold
Number/Modular/	25 / 24k / 1228	modular forms, Φ_{41} , CM-163, $j(\tau_{163})$
Number/Hypergeom./	3 / 5k / 173	Clausen, Picard–Fuchs Wronskian, Chowla–Selberg; Ramanujan reduction
CTMC/	7 / 16k / 772	DTMC/CTMC, density process, Doob bound
Kurtz/	4 / 1.8k / 51	three versions of Kurtz’s mean-field theorem
<i>majority population-protocol formalizations (in progress / companion packages)</i>		
PopulationProtocol/	71 / 48k / 1204	<i>stable exact majority</i> (Doty et al.) — deterministic chain largely done, probabilistic time bound in progress
pp-proof/	24 / 6.7k / 221	<i>simple approximate majority</i> [1] — $O(n \log n)$ convergence; central-region drift gap found and fixed; near complete
SSExtactMajority/	67 / 95k / 1999	<i>silent self-stabilizing exact majority</i> [13] — in progress
Total	≈ 302 / $\approx 369k$ / $\approx 10.6k$	the axiom-clean Ripple core ($\approx 270k$ lines) plus the companion majority packages

Figure 1: Map of the Ripple framework. “defs” counts top-level **theorem/lemma/def** declarations. The upper block is the completed, axiom-clean core development. The lower block groups the three *majority* population-protocol formalizations: the Doty et al. stable exact-majority protocol (PopulationProtocol/, inside Ripple), and two companion packages — the Angluin–Aspnes–Eisenstat *simple approximate majority* convergence proof (pp-proof/, near complete; this is where gap A of §4 was found and fixed) and the Kanaya et al. (SSS 2025) silent self-stabilizing exact-majority protocol. The two exact-majority efforts are still under active development and not yet building cleanly. They make the point concrete: the open framework is already carrying new threads.

Foundational infrastructure (reusable, and new to Lean). A large part of Ripple is general-purpose and independent of any particular target number. It includes the GPAC/PIVP core and the single CertifiedBoundedTimeComputable type (§3) that pins down what “computes α ” means; the dual-rail compiler; the four-stage LPP compilation pipeline; and—built from the ground

up—a continuous-time Markov chain layer (DTMC, CTMC, the density process) together with three machine-checked versions of *Kurtz’s mean-field theorem*. These are foundational definitions and theorems: a verified CTMC theory and a formal Kurtz limit did not previously exist in Mathlib, and they are reusable far beyond this paper — by anyone formalizing stochastic chemical kinetics, population dynamics, or mean-field limits, whether or not they care about computable numbers.

Two threads of research. On top of that foundation we carry two threads. The first is the theory of *CRN-computable real numbers* — from the foundational constants through $\zeta(3)$ (§5), the π obstruction (§6), and the LPP compilation pipeline — and is the line this document develops in most detail. The second is *population protocols and majority*: the approximate-majority convergence proof and the two exact-majority protocols of Figure 1. Neither thread is yet a *complete* survey of its area — both remain in progress — but in each the bulk of the supporting infrastructure is already in place. That is the point: the framework is general enough to carry both, and to carry more.

Built to extend. Because the CRN-computable class is a Lean type, adding a new computable number is a plug-in: supply a PIVP, prove boundedness and an exponential modulus, invoke the pipeline; a definition proved once is reused everywhere, the type system guaranteeing the meaning never silently drifts between theorems written years apart. More broadly, the framework is open in the strong sense: the foundational layer (the GPAC core, the CTMC/Kurtz theory, the compilation pipelines) is a base on which others can mount entirely new threads of research, or which they can take piecemeal for their own.

3 The Model Ladder, in Depth

This section develops the model ladder sketched in the introduction into a self-contained account of what Ripple formalizes and how the pieces connect. We treat, in order, the analog model and its history (§3.1), the notion of CRN-computability (§3.2), the foundational constants (§3.3), Catalan’s constant (§3.4), the LPP compilation pipeline (§3.5), the stochastic continuous-time-Markov-chain (CTMC) layer with Kurtz’s mean-field theorem (§3.6), and finally the LPP→Kurtz connection — the interchange of the population limit and the time limit (§3.7), which the senior author regards as the conceptual keystone of the whole construction.

Throughout, we use the following notation: a CRN in its mass-action limit is a polynomial ODE system $\dot{x}_i = p_i(x) - x_i d_i(x)$ with p_i, d_i having non-negative coefficients; dropping the sign structure gives a PIVP $\dot{y} = p(y)$; and “ α is CRN-computable” means a coordinate satisfies $|y_k(t) - \alpha| \leq Ce^{-t}$.

3.1 The analog model: GPAC, PIVP, and the mass-action CRN

Shannon’s GPAC. The continuous model underneath everything is Shannon’s *General Purpose Analog Computer* (GPAC), introduced in 1941 as an idealization of Vannevar Bush’s differential analyzer. A GPAC is a circuit of four primitive elements — constant, adder, multiplier, and integrator — wired together. Shannon proved that the functions a GPAC can generate are exactly the *differentially algebraic* functions: those satisfying some polynomial differential equation. Equivalently, a GPAC computes precisely the components of a solution to a *polynomial initial value problem* (PIVP)

$$\dot{y} = p(y), \quad y(0) = y_0 \in \mathbb{Q}^d, \quad p \in \mathbb{Q}[y_1, \dots, y_d]^d.$$

This was long believed to be strictly weaker than Turing computation (the Gamma function is not differentially algebraic, and so escapes the classical GPAC), but Bournez, Campagnolo, Graça, and Hainry showed in 2007 that a suitable real-analysis formalization of the GPAC computes all computable functions on computable compact intervals, and is in fact polynomial-time equivalent to the Turing machine. The polynomial vector field is universal.

From voltages to concentrations. A chemical reaction network is the same mathematics with “concentration” in place of “voltage”. A reaction $A + B \rightarrow C + D$ under mass-action kinetics fires at a rate proportional to the product of reactant concentrations, and the induced rate of change of each species is a polynomial. The mass-action CRN is therefore a PIVP wearing a sign constraint: production terms are non-negative polynomials, and a species can only be degraded if it is present, which forces every negative monomial in $\dot{x}_i$ to carry x_i as a factor. This is the Hárs–Tóth “lack of negative cross-effects” condition, and it is exactly the gap between an arbitrary PIVP and a chemically realizable one.

Equivalence and realizability. The equivalence between PIVPs and mass-action CRNs is classical but not free: it is the realizability question of reaction kinetics. Hárs and Tóth (1979) proved that a polynomial ODE $\dot{x} = P(x)$ is realizable by a mass-action CRN if and only if it satisfies the no-negative-cross-effect condition, and gave an explicit *canonic mechanism* — one reaction per monomial term — that realizes it. Finding a CRN is mechanical; finding a *minimal* one is a network-flow / matching problem that becomes genuinely hard under structural constraints (fewest reactions, weak reversibility, or the “no self-production” constraint that governs population-protocol constructions). For the purposes of Ripple, the relevant fact is the easy direction: every bounded PIVP that arises in the constructions below can be split into a production polynomial p_i and a degradation polynomial d_i with the right signs, and this split is what Lean’s `PolyCRNDecomposition` structure records.

3.2 CRN-computability

A real number α is **CRN-computable** if some PIVP with rational data has a coordinate $y_k(t)$ that stays bounded and satisfies the explicit exponential bound

$$|y_k(t) - \alpha| \leq C e^{-t} \quad \text{for all } t \geq 0.$$

This is the single notion of computability used throughout: a bounded polynomial system whose output coordinate converges to the target with a certified exponential modulus.

One shared Lean definition. The reason the formalization is worth doing is that the four source papers (§3) use the phrase “bounded CRN computes α ” in subtly different informal senses. In Lean there is exactly one carrier:

$$\text{CertifiedBoundedTimeComputable } (d : \mathbb{N}) (\alpha : \mathbb{R})$$

(`Ripple/Core/BoundedTime.lean`), a structure bundling a PIVP of dimension d , a designated output coordinate, a boundedness witness, and an explicit exponential-convergence modulus to α . Every theorem in the project — the constant constructions, the dual-rail compiler, the LPP pipeline — consumes or produces this one structure, so the meaning of “computes α ” is type-checked to be identical across results written years apart. A forgetful coercion drops the certificate when only the weaker existence statement (`BoundedTimeComputable`) is wanted.

3.3 Foundational constants

Ripple re-formalizes the elementary foundational constants — e , π (through $\pi/4 = \arctan 1$), $\ln 2$, $\frac{1}{2}e^{-1}$, the Euler–Mascheroni γ (through $\gamma = 1 - \Gamma'(2)$), and the Dottie number — each as a bounded PIVP with an explicit exponential modulus, all axiom-free. These are linearly computable targets already settled in Huang et al.’s earlier work [12]; the only reciprocal any of them needs is the bounded state $1/(1+t)$ (satisfying $r' = -r^2$), which is what keeps them inside the polynomial-PIVP class. We pass over them; the one shown below in compact form, Catalan’s constant, is included only as the worked example of that integral-to-PIVP recipe.

3.4 Catalan’s constant G

As the compact worked example of that recipe, Catalan’s constant $G = \sum_{n \geq 0} (-1)^n / (2n+1)^2 \approx 0.9160$ has the Laplace-type representation $G = \int_0^\infty \frac{s e^{-s}}{1+e^{-2s}} ds$. Its only non-polynomial factor is $1/(1+e^{-2s})$; carrying that as a state $W := 1/(1+e^{-2t})$, which satisfies a quadratic ODE (the substitution of [11, Cor. 19]), turns the whole thing into the four-state polynomial PIVP

$$\begin{cases} E' = -E, \\ R' = E - R, \\ W' = 2E^2 W^2, \\ G' = R \cdot W, \end{cases} \quad (E, R, W, G)(0) = (1, 0, \tfrac{1}{2}, 0),$$

with closed forms $E = e^{-t}$, $R = t e^{-t}$, $W = 1/(1+e^{-2t})$, so $G(t) \rightarrow G$ with the modulus $|G(t) - G| \leq (t+1)e^{-t}$. This is the theorem `catalan_is_lpp_computable` (`Ripple/Number/CatalanCertified.lean`), verified clean. It rests on the same generic certificate machinery — a bounded PIVP plus an exponential modulus — that every target in Ripple ultimately uses.

3.5 The LPP compilation pipeline

The *large-population protocol* (LPP) model is the natural weakening of the CRN model in which populations are bounded: instead of arbitrary real concentrations, one tracks fractions $x_i = n_i/N$ of a fixed population of size N , and the dynamics are constrained accordingly. The central compilation result is the theorem `bounded_crn_is_lpp_computable_unconditional` (in `Ripple/LPP/BoundedLPP.lean`), verified clean, which states that *every* bounded certified PIVP can be compiled into an LPP computing the same number. It is strictly stronger than the original [11] statement, because it removes the paper’s implicit hypothesis that the certified computation stays inside $[0, 1]$ at all times — a hypothesis the formalization showed can fail, since the output coordinate may transiently overshoot while converging from above (see the gap discussion §4(A)). The repair is a saturating low-pass filter $y' = (x - y)(U - y)$ with $U \in (\alpha, 1) \cap \mathbb{Q}$, which clamps the output into a user-chosen interval without changing the limit.

The four stages. The compiler is a four-stage pipeline, each stage a separately verified transformation of the certified system:

- (i) **Quadraticization.** An arbitrary-degree polynomial vector field is reduced to a degree- ≤ 2 one by introducing auxiliary product variables $z = x_i x_j$ with their own ODEs. This is the standard GPAC reduction; it preserves the solution and the convergence modulus, at the cost of more coordinates.

- (ii) **Bound-to-small- λ closure.** The bounded system is rescaled in time / rate so that all reaction rates fit under the LPP rate budget (the “small- λ ” regime), using the boundedness certificate to control the rescaling.
- (iii) **Triple-product discretization.** The continuous quadratic dynamics are realized by population-level interactions; the discretization step matches the mass-action products to the bimolecular interaction rates of a population of size N .
- (iv) **Pure-LPP embedding.** The discretized system is embedded into the formal LPP model, producing the `IsLPPComputable` witness.

Algebraic specialization. A clean corollary is that every algebraic $\alpha \in [0, 1]$ is LPP-computable, obtained by starting from its minimal polynomial: the quadratic $\dot{x}_1 = \varepsilon(a_0 + a_1x_1 + a_2x_1^2)$ with the target root as a stable fixed point feeds the pipeline directly. The formalization here also exposed and repaired a gap in the classical Bournez–Fragliaud–Koeqler conservation trick (§4(B)): the balancing variable $\dot{x}_\delta = -\sum_j \dot{x}_j$ is not CRN-implementable as written, and Ripple’s balancing-dilation repair recovers it.

3.6 The CTMC layer and Kurtz’s mean-field theorem

Everything above treats CRNs as deterministic dynamical systems. But a real CRN with N molecules is a *stochastic* process: a continuous-time Markov chain (CTMC) whose state counts molecules of each species, jumping by a reaction’s net-change vector at the mass-action rate. The deterministic ODE is what survives in the limit $N \rightarrow \infty$, and the theorem making this precise is due to Kurtz (1970). Ripple’s `Ripple/CTMC/` and `Ripple/Kurtz/` layers formalize this bridge, all sorry-free.

The density process and martingale decomposition. The core object is the **density process** $\bar{X}^N(t) = X(t)/N$, where $X(t)$ is the molecule-count vector and N the system size. The file `Ripple/CTMC/DensityDependent.lean` builds it up from discrete-time and continuous-time chains, and establishes the central *martingale decomposition*

$$\bar{X}^N(t) = \bar{X}^N(0) + \int_0^t F(\bar{X}^N(s)) ds + M^N(t),$$

where F is the drift (the mean-field vector field) and M^N is a martingale satisfying the quadratic-variation bound

$$\mathbb{E} \left[\sup_{s \leq t} \|M^N(s)\|^2 \right] \leq \frac{Ct}{N}.$$

A subtle measure-theoretic obstruction surfaced here and is resolved in `RandomIndexDoob.lean`: applying Doob’s L^2 inequality at the random jump-count index requires a martingale measurable with respect to the *shifted* filtration $G_n = F_{n+1}$, not the filtration F_n the martingale natively lives on. The fix defines $\tilde{M}(n) = M(n+1)$, proves it is a G -martingale, and applies Doob to $\tilde{M}$. This is the kind of step that has no informal analogue — it exists only because the machine refuses to apply Doob to a martingale on the wrong filtration.

Three versions of the theorem. The directory `Ripple/Kurtz/` contains three sorry-free versions of the mean-field limit, in increasing strength.

Weak (convergence in probability). The theorem `kurtz_mean_field_convergence` states that for every $\varepsilon > 0$,

$$\mathbb{P}\left(\sup_{t \leq T} \|\bar{X}^N(t) - x(t)\| > \varepsilon\right) \longrightarrow 0 \quad \text{as } N \rightarrow \infty,$$

where $x(t)$ solves the mean-field ODE $x' = F(x)$. In Lean (`Ripple/Kurtz/MeanField.lean`) the theorem takes a family of density processes, a mean-field solution, convergence of the initial conditions in probability, and a packaged Grönwall–Markov hypothesis, and concludes the corresponding `Tendsto` to 0.

Strong (almost sure rate), `kurtz_strong_approximation`. With probability one,

$$\sup_{t \leq T} \|\bar{X}^N(t) - x(t)\| = O\left(\frac{\log N}{\sqrt{N}}\right),$$

stated in Lean as a limsup bound on $(\sqrt{N}/\log N) \cdot \sup_t \|\cdot\|$, driven by a strong-coupling / Azuma–Hoeffding + Borel–Cantelli hypothesis `h_as_bound`.

CLT-scale second moment, `kurtz_clt_second_moment`.

$$\mathbb{E}\left[\sup_{t \leq T} \|\bar{X}^N(t) - x(t)\|^2\right] \leq \frac{C}{N},$$

the quantitative refinement that the fluctuations are $O(1/\sqrt{N})$.

Sketch of the weak proof. The chain (assembled in `kurtz_convergence_for_density_dep_ctmc`) is three steps glued together.

1. *Grönwall.* Subtracting the ODE from the martingale decomposition gives $\bar{X}^N(t) - x(t) = (\bar{X}^N(0) - x_0) + \int_0^t [F(\bar{X}^N(s)) - F(x(s))] ds + M^N(t)$. With F being L -Lipschitz, the integral Grönwall lemma (`integral_gronwall_core` in `Ripple/Kurtz/IntegralGronwall.lean`, proved from Mathlib’s derivative-form `norm_le_gronwallBound_of_norm_deriv_right_le`) turns this into the pathwise bound $\sup_{t \leq T} \|\text{error}(t)\| \leq \sup_{s \leq T} \|M^N(s)\| \cdot e^{LT}$ for almost every ω (`h_gronwall_pw_of_density_process`).
2. *Martingale L^2 bound.* The `DensityProcess` interface supplies $\mathbb{E}[\sup \|M^N\|^2] \leq CT/N$ (the bound from the decomposition above).
3. *Markov.* `martingale_markov_bound` converts the L^2 bound into a probability bound $\mathbb{P}(\sup \|M^N\| \geq \delta) \leq CT/(N\delta^2)$, and `kurtz_chebyshev_bound` / `kurtz_gm_of_event_inclusion` assemble the event inclusion $\{\sup \text{error} > \varepsilon\} \subseteq \{\text{init error} > \delta\} \cup \{\sup \|M^N\|^2 \geq \delta^2\}$ into the $O(1/N)$ right-hand side that $\rightarrow 0$.

The strong version additionally invokes Azuma–Hoeffding and Borel–Cantelli from Mathlib (via a Grönwall exponential-tail bound).

3.7 The LPP→Kurtz connection: interchanging two limits

This is the conceptual keystone. An LPP is supposed to “compute α ” — but *what does that mean*, given that an LPP is a finite- N stochastic object while α is a real number reached only in a limit? The answer is that an LPP computes α through **two limits, taken in a definite order**:

$$\underbrace{\text{stochastic PP at size } N}_{\text{finite, random}} \xrightarrow[N \rightarrow \infty]{\text{Kurtz}} \underbrace{\text{mean-field ODE } x(t)}_{\text{deterministic}} \xrightarrow[t \rightarrow \infty]{} \underbrace{\alpha}_{\text{the target}}.$$

The $N \rightarrow \infty$ limit (population limit) is justified by Kurtz’s theorem of §3.6: the stochastic protocol converges to the deterministic mean-field ODE. The $t \rightarrow \infty$ limit (time limit) is the ODE coordinate converging to the target, exactly the CRN-computability certificate of §3.2. Establishing LPP-computability rigorously means *controlling the order* of these two limits — it is not automatic that $\lim_t \lim_N = \lim_N \lim_t$, and the piece that makes the two compatible is what was added in the Kurtz layer (this is the part “added later” on top of the original deterministic constructions).

The order of limits, and the hypotheses that make it safe. The two iterated limits genuinely need not agree, so it is worth saying exactly what extra is required. Kurtz’s theorem is a statement on *compact* time intervals: for each fixed T , $\sup_{t \leq T} \|\bar{X}^N(t) - x(t)\| \rightarrow 0$ in probability as $N \rightarrow \infty$, where $x(\cdot)$ solves the mean-field ODE. It says *nothing* as $T \rightarrow \infty$. Consequently $\lim_N \lim_t$ (run the finite- N chain to its equilibrium, *then* enlarge the population) and $\lim_t \lim_N$ (pass to the ODE, *then* run it to its fixed point) can differ — the finite chain can, for instance, fluctuate into an absorbing boundary state the ODE never visits. The interchange becomes valid under two hypotheses beyond the Lipschitz drift that Kurtz already needs:

1. **Exponential stability of the limit ODE.** $\dot{x} = F(x)$ has a unique equilibrium x^* in the invariant region that is globally exponentially attracting, $\|x(t) - x^*\| \leq Ce^{-ct}$ — which is exactly the exponential-modulus (CRN-computability) certificate of §3.2 applied to the output coordinate, $x_{\text{out}}^* = \nu$.
2. **Uniform-in- N concentration.** The finite- N family is drawn to x^* *uniformly* in N : for every $\varepsilon > 0$ there exist T, N_0 with $\mathbb{P}(\|\bar{X}^N(t) - x^*\| > \varepsilon) < \varepsilon$ for all $t \geq T$ and $N \geq N_0$. A uniform Foster–Lyapunov / exponential-ergodicity bound supplies this; it is what forbids late-time escape and is the substantive content the compact-interval Kurtz theorem does not provide.

The honest theorem is therefore the conjunction — “Lipschitz drift $\Rightarrow$ Kurtz on compacts, and (1)–(2) $\Rightarrow$ the two iterated limits coincide and both equal ν ” — not a single black-box equality $\lim_t \lim_N = \lim_N \lim_t$.

The formalized objects, in Lean. The specialization of Kurtz’s theorem to population protocols lives in `Ripple/Kurtz/PopulationProtocol.lean`. A population protocol is a finite set of reactions $S_i + S_j \rightarrow S_k + S_\ell$ (`PPReaction`, `PopProtocol`), each with net-change vector `PPReaction.netChange` and mass-action rate `PPReaction.massActionRate` $= x_i x_j$. The bridge proceeds through these formalized objects.

Population $\rightarrow$ rate spec. `toRateSpec` packages the protocol as a density-dependent `RateSpec`: the jump set is the image of `netChange`, and the per-direction rate is the sum of mass-action rates of reactions with that net change. The fields `rate_nonneg`, `rate_support`, and `rate_lipschitz` are all discharged — in particular the drift is globally Lipschitz, which is exactly the hypothesis Kurtz needs. Conservation of total population is `PPReaction.netChange_sum_zero` and `toRateSpec.conservative_jumps`.

The mean-field ODE. `meanFieldDrift` is the quadratic vector field

$$x'_r = \sum_{\text{reactions}} (\text{net production of } S_r) \cdot x_{\text{in}_1} x_{\text{in}_2},$$

and `meanFieldDrift_eq_drift` proves it coincides with the drift extracted from `toRateSpec`. This is the ODE that Kurtz’s theorem produces as the $N \rightarrow \infty$ limit.

The ODE is a Ripple PIVP. `toPolyPIVP` builds a degree- ≤ 2 PolyPIVP (with rational, in fact integer, coefficients from the stoichiometry) from the protocol, and `toPolyPIVP_field_eq` proves its semantic field equals `meanFieldDrift`. Moreover `meanField_isCRNImplementable` produces a PolyCRNDecomposition, certifying the mean-field ODE is itself mass-action realizable — which it must be, since it came from mass-action kinetics.

The constructive density-process bridge. A constructive `DensityProcess` is built for input-distinct protocols, supplying conservative jumps and simplex-boundary compatibility internally and leaving only the generator-residual Doob/bracket inequality as a stochastic input. This is the object that `kurtz_mean_field_convergence` consumes.

The interchange, stated honestly. The two-limit interchange is captured by the definition `Computes` and the theorem `computes_implies_pivpComputable`. The definition says a protocol *computes* ν if its mean-field ODE converges to ν along the output coordinate:

$$\begin{aligned} \text{Computes } \text{pp output } \nu &:= \exists x_0 : \text{Fin } d \rightarrow \mathbb{Q}, \forall \text{sol}, (\text{sol } 0 = x_0) \rightarrow \\ &(\text{sol solves the mean-field ODE}) \rightarrow \text{sol}(t)_{\text{output}} \xrightarrow[t \rightarrow \infty]{} \nu. \end{aligned}$$

Here the $N \rightarrow \infty$ limit has *already* been taken — the hypothesis is phrased about the deterministic ODE solution, which Kurtz’s theorem (`kurtz_mean_field_convergence`) identifies as the population limit of the stochastic protocol. The theorem `computes_implies_pivpComputable` then takes the $t \rightarrow \infty$ limit: it shows that if `pp` computes ν , then ν is PIVP-computable (hence CRN-computable), by exhibiting `pp.toPolyPIVP` as the witnessing PIVP and transporting the time-limit convergence through `toPolyPIVP_field_eq`.

We are deliberately precise about the division of labor, because it is easy to overclaim here. What Ripple proves is each *link* of the chain separately and `sorry-free`:

1. the population limit $N \rightarrow \infty$ (stochastic PP $\rightarrow$ mean-field ODE), via the Kurtz theorems of §3.6 applied to `toRateSpec` and the constructive density process;
2. the identification of that limit ODE with a Ripple PIVP (`meanFieldDrift_eq_drift`, `toPolyPIVP_field_eq`, `meanField_isCRNImplementable`);
3. the time limit $t \rightarrow \infty$ of the ODE coordinate, and its transport to PIVP-computability (`computes_implies_pivpComputable`).

The interchange-of-order content is encoded structurally: `Computes` is defined on the *post*-population-limit ODE, and Kurtz supplies the guarantee that this ODE is the correct $N \rightarrow \infty$ object, so that proving the $t \rightarrow \infty$ convergence of the ODE coordinate legitimately certifies the finite- N protocol’s computational target. A single monolithic theorem asserting $\lim_t \lim_N = \lim_N \lim_t$ as one statement is *not* what the file contains; rather, the file factors that statement into the three verified links above, which is the honest and reusable form of the claim. In terms of the two hypotheses just stated, Ripple establishes (1) — the exponential $t \rightarrow \infty$ convergence of the ODE output coordinate — together

with the Lipschitz drift and the compact-interval Kurtz step; hypothesis (2), the uniform-in- N late-time concentration, is the one it does *not* discharge. The `Computes` definition is stated on the post- $N \rightarrow \infty$ ODE precisely so that this uniform bound is bracketed as an explicit modeling assumption rather than silently asserted. The file’s own closing comment names this explicitly: “this is the formal justification for why Ripple’s PIVP framework captures exactly what population protocols compute in the large- N limit.”

The upshot of the connection. The deterministic constructions (§§3.3–3.4) and the LPP pipeline (§3.5) live entirely on the ODE side. The CTMC/Kurtz layer (§3.6) is what connects that ODE side back to the *physical* stochastic CRN with finitely many molecules. The LPP→Kurtz connection of this subsection is the seam between the two: it is the formal statement that the number an LPP computes — defined through finite- N stochastic dynamics — is exactly the number the corresponding PIVP converges to, with the order of the population and time limits controlled. Without it, “the LPP computes α ” and “the PIVP converges to α ” would be two unrelated assertions; with it, they are the same assertion, type-checked.

4 Gaps Exposed by Formalization

A machine-checked proof has no human reader to fill in the routine steps. Every logical step must be justified, every case covered, every interaction pattern analyzed. In the course of formalizing the constructions that make up the Ripple pipeline — both new constructions and the published results they build on — this discipline has surfaced a number of genuine gaps. In each case the published *theorem* is correct; the gap lives in a proof sketch, an intermediate construction, or an unstated hypothesis, and surfaces in Lean as a compile error or as an explicit counterexample rather than as a footnote.

We document three representative gaps in detail below — the mathematics of the original claim, the precise reason it fails or is incomplete, the explicit counterexample where one exists, and the fix the formalization carries — and note that the formalization turned up others beyond these (for instance an interior-saddle counterexample to a 2024 algebraic-CRN basin-entry step, and an atomic-initialization subtlety in the exact-majority error-jump paths).

The three gaps detailed here.

- (A) §4.1 — Angluin–Aspnes–Eisenstat (2008) approximate majority: the central-region multiplicative drift inequality is *false*; explicit $n = 4$ counterexample; product-form supermartingale fix. (The final $O(n \log n)$ theorem is correct.)
- (B) §4.2 — the large-population-protocol [11] main theorem: the certified CRN output can transiently leave $[0, 1]$; saturating low-pass filter fix.
- (C) §4.3 — reaching algebraic numbers of *arbitrary degree* in the urn model needs the non-existence of solutions to $a^x - b^y = 1$ (Catalan’s conjecture); the classical Cassels (1960) descent supplies only *divisibility*, pinning down exactly which case the argument must still close.

4.1 (A) The approximate-majority central-region drift inequality is false

This is the headline gap. The 3-state approximate majority protocol of Angluin, Aspnes, and Eisenstat (*Distributed Computing*, 2008) is one of the most studied population protocols, and its $O(n \log n)$ convergence proof is a classic. Formalizing that proof in Lean 4 (the `pp-proof` repository, $\approx 6,650$ lines, zero `sorry`) revealed that one intermediate step in the original proof sketch does not

hold as stated. The final theorem is correct; the central region needs a different argument from the one the sketch suggests.

The protocol. There are n agents, each in one of three states: x (opinion A), y (opinion B), or b (blank/undecided). At each step two agents are chosen uniformly at random and updated by

$$xb \rightarrow xx, \quad yb \rightarrow yy, \quad xy \rightarrow xb \text{ or } xy \rightarrow yb.$$

Blanks adopt the opinion they meet; opposing opinions cancel into a blank. The theorem is that the protocol reaches consensus in $O(n \log n)$ interactions with high probability, matching the information-theoretic lower bound.

The potential and the four-region partition. The proof uses the potential function

$$f = u^2 + 2n, \quad u = x - y \quad (\text{the opinion gap}).$$

The configuration space is partitioned into four regions by which count is large (threshold $7n/8$):

$$\text{Large-}x \ (x \geq \tfrac{7n}{8}), \quad \text{Large-}y \ (y \geq \tfrac{7n}{8}), \quad \text{Large-}b \ (b \geq \tfrac{7n}{8}), \quad \text{Central} \ (\text{no count} \geq \tfrac{7n}{8}).$$

For the three *corner* regions the proof establishes a multiplicative drift: the expected value of $1/f$ contracts by a factor $(1 - \delta)$ per step, $\delta = \Theta(1/n)$, which gives geometric decay by standard arguments.

The suggested inequality. The paper's Lemma 4 concerns the central region. The surrounding proof sketch implicitly carries the same multiplicative-drift framework into the central region, i.e. it needs, for $f' = f(C_{t+1})$ the potential after one step,

$$\mathbb{E}\left[\frac{1}{f'} \mid \mathcal{F}_t\right] \leq (1 - \delta) \frac{1}{f_t} \quad (\text{central region}). \quad (1)$$

Inequality (1) does not hold in general.

The counterexample ($n = 4$). Take $n = 4$, with counts $x = 1$, $b = 0$, $y = 3$. This is a central configuration: the threshold is $7 \cdot 4/8 = 3.5$, and the largest count is $y = 3 < 3.5$. The opinion gap is $u = x - y = -2$, so the potential is

$$f = (1 - 3)^2 + 2 \cdot 4 = 4 + 8 = 12.$$

Enumerating all unordered pairs of agents and their post-interaction potentials, one computes

$$\frac{\mathbb{E}[1/f']}{1/f} = \frac{103}{102} > 1. \quad (2)$$

So $1/f$ *increases* on average from this configuration — the opposite of the contraction (1) requires (which would need the ratio $\leq 1 - \delta < 1$). Truncating to the active central region does not rescue the estimate: *all* transitions out of $(1, 0, 3)$ remain in the central region, so there is no absorption boundary to provide the missing correction. The counterexample is not an artifact of small n : it was verified computationally to persist for every n from 4 through 50.

The reason it fails: Jensen runs the wrong way. The function $x \mapsto 1/x$ is *convex* on $(0, \infty)$. Even when the expected drift of f is non-negative, $\mathbb{E}[\Delta f] \geq 0$, Jensen’s inequality applied to a convex function gives

$$\mathbb{E}\left[\frac{1}{f'}\right] \geq \frac{1}{\mathbb{E}[f']} \geq \frac{1}{f}, \quad (3)$$

which points in the *opposite* direction to (1). The inequality one wants is fighting the convexity of $1/x$. In the corner regions the step size of f is small relative to f itself, so the convexity (second order) error is negligible and the first-order drift wins. In the central region this is no longer true: a single xy interaction changes u by ± 1 , moving f by an amount of order

$$|\Delta f| \sim |(u \pm 1)^2 - u^2| = |2u \pm 1| \approx 2|u| + 1,$$

which is *comparable* to $f = u^2 + 2n$ itself (in the example, a step of order $2 \cdot 2 + 1 = 5$ against $f = 12$). When the step size is comparable to the value, the convexity term in (3) dominates and the naive drift estimate is simply false.

The fix — a product-form supermartingale. The correct argument — which is in fact the content of the paper’s own Lemma 4, though the sketch never makes the link to the drift framework explicit — works with an *augmented state* that tracks cumulative interaction counts rather than reciprocals of the potential. Let S_t^{vb} and S_t^{xy} count, up to time t , the cumulative number of vb -type interactions (a blank adopting an opinion) and xy -type interactions (two opinions cancelling). On the augmented state $(C_t, S_t^{vb}, S_t^{xy})$ define

$$M_t = \frac{\alpha_{vb}^{S_t^{vb}} \alpha_{xy}^{S_t^{xy}}}{f(C_t)}, \quad \alpha_{vb} = \frac{16n + 7}{16n}, \quad \alpha_{xy} = \frac{16n - 5}{16n}. \quad (4)$$

The point of the multiplicative weights α_{vb}, α_{xy} is that each interaction type pays a fixed factor, so the supermartingale *factors through the interaction types* instead of averaging reciprocals — this is what sidesteps the convexity problem entirely. The per-step supermartingale condition $\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] \leq M_t$ reduces, after clearing denominators, to two purely algebraic inequalities (here v is the count of the opinion adopted in a vb interaction):

$$(VB) \quad (16n + 7) f(v(f + 1) - 2u^2) \leq 16n v((f + 1)^2 - 4u^2), \quad (5)$$

$$(XY) \quad (16n - 5) f(f + 1) \leq 16n((f + 1)^2 - 4u^2). \quad (6)$$

Both hold for *all* $n \geq 1$ — not merely for “sufficiently large n ” — and both were verified formally. From the supermartingale (4) the geometric decay then follows in every region by the optional-stopping / drift machinery, with the central region now correctly handled.

The formalized fix. The pp-proof development encodes the entire chain from these algebraic lemmas to the final geometric-decay bound. The two regional theorems read, in Lean, with a uniform decay rate $1 - 1/(15000n)$:

$$\text{prob_in_activeCentral_le}, \quad \text{prob_in_activeLargeX_le}$$

(and analogues for Large- y , Large- b), each bounding $(\text{kernel}^t c_0)(\text{active region})$ by $C \cdot \text{ofReal}((1 - 1/(15000n))^t) \cdot \text{potentialTrunc}(c_0)$. Two engineering points are worth recording. First, the probability bounds live in $\mathbb{R}_{\geq 0}^\infty$ (Mathlib’s `ENNReal`) while the algebraic core lives in $\mathbb{R}$; bridging the two is delicate because `ENNReal` is not a field — one must use `ENNReal.mul_inv` with explicit finiteness/nonzero

side conditions, `ENNReal.inv_pow` in place of the generic `inv_pow`, and route natural-number casts through `ENNReal.ofReal_natCast`. Second, the constant 15000 is deliberately looser than the paper’s tighter implicit constants: the extra margin lets the formal inequalities (5) and (6) close without razor-thin case analysis, at no cost to the asymptotic $O(n \log n)$ bound. The development is roughly 6,650 lines across 23 files, axiom-free.

Honest scope. The gap is in the proof sketch, not in the result: the published $O(n \log n)$ theorem is correct. The repair is already implicit in the authors’ own Lemma 4; formalization made the disconnect between Lemma 4 and the surrounding drift narrative explicit, and supplied the all- n algebraic inequalities that the central region needs.

4.2 (B) The LPP main theorem can leave $[0, 1]$

This gap is in the large-population-protocol (LPP) large-population-protocol main theorem [11]. The LPP model is a natural weakening of the CRN model in which populations are bounded, and the main theorem states that every bounded certified PIVP can be compiled into an LPP computing the same real number.

The unstated assumption. As originally written, the theorem assumed that the certified CRN computation stays inside $[0, 1]$ *at all times*. This is the natural setting for a population protocol: a coordinate is read as a fraction of the population, so it must lie in $[0, 1]$. The trouble is that “the certified CRN converges to $\alpha \in [0, 1]$ with exponential rate” does not imply “the output coordinate never leaves $[0, 1]$.” A coordinate converging to α *from above* can transiently overshoot 1 (or, symmetrically, dip below 0) by a small amount before settling. In paper form this is at most a footnote; in Lean it is a genuine compile error, because the embedding step requires the coordinate to be a valid population fraction at *every* time, not merely in the limit.

The fix — a saturating low-pass filter. The repair is to post-process the (possibly overshooting) output coordinate x through a saturating low-pass filter

$$y' = (x - y)(U - y), \quad U \in (\alpha, 1) \cap \mathbb{Q}. \quad (7)$$

The factor $(x - y)$ is the ordinary low-pass tracking term — y relaxes toward x — and the additional factor $(U - y)$ is a saturation gate: as y approaches the user-chosen ceiling $U < 1$, the gate $(U - y)$ shrinks the drive to zero, so y cannot cross U , and in particular cannot leave $[0, 1]$. Since $U > \alpha$, the equilibrium of the filtered system still has $y \rightarrow \alpha$: the saturation never engages near the limit, so it clamps the output into a user-chosen interval *without changing the value computed*. The ceiling U is rational, keeping the filtered system inside the rational-coefficient PIVP category.

The formalized fix. The formalized compilation theorem (§3.5) incorporates the filter (7) and is *strictly stronger* than the original paper statement: it drops the always-in- $[0, 1]$ hypothesis and proves the compilation unconditionally for every bounded certified PIVP. The compilation itself is the four-stage pipeline — quadraticization, bound-to-small- λ closure, triple-product discretization, and pure-LPP embedding — and the filter is what makes the final embedding step go through in all cases. This is the cleanest illustration in the project of formalization turning an implicit modeling assumption into an explicit, provable, and generically applicable repair.

4.3 (C) Arbitrary-degree algebraic numbers in the urn model: a Catalan-conjecture dependency

This last item situates a population-protocol construction against a classical number-theoretic lemma, and is about being honest about exactly what that lemma supplies. The connection to population protocols is direct. The *urn model* (Albenque–Gerin, 2012) is a population protocol in the strict sense: n agents each carry one of two colours, at each step a fixed number k of them are sampled and all recoloured by a majority-type rule, and the proportion of one colour converges, as $n \rightarrow \infty$, to a fixed point of a Bernstein polynomial — exactly the large-population-protocol notion of *computing* a number in $[0, 1]$ (§3.5), specialized to a single two-colour species and k -way interactions. This submodel computes a dense set of algebraic numbers (while excluding almost every rational), and the headline claim is that it reaches algebraic numbers of *arbitrary degree*. That claim is what carries the gap: its proof runs, at one step, into a *Catalan-type* Diophantine question — it needs the *non-existence* of solutions to the equation $a^x - b^y = 1$ (beyond the lone Catalan exception $3^2 - 2^3 = 1$), i.e. Catalan’s conjecture (Mihăilescu’s theorem). To pin down exactly what is available toward that, we formalized the classical Cassels (1960) elementary descent for the equation: 45 kernel-clean declarations, zero **sorry**.

The formalization clarified a point easy to blur in prose, and the point is not small. Cassels’ elementary descent does *not* prove non-existence of solutions; what it yields is a *divisibility* conclusion — in a putative solution one obtains $p \mid y$ and $q \mid x$ for the relevant primes p, q . Promoting that divisibility to outright non-existence is exactly the strength of **Catalan’s conjecture**, settled only in 2002 by Mihăilescu’s theorem [15]; the elementary descent does not deliver it. By formalizing the descent we pinned down *precisely* the residual case — the step from “the primes divide as stated” (which Cassels gives) to “no solution exists” (which the urn-model argument needs) — and showed it to be that deep theorem, not a routine lemma. The urn model’s arbitrary-degree result is therefore sound (Catalan’s conjecture is now a theorem), but it rests, at this one step, on a genuinely hard result that the elementary presentation hides — a dependency only the formalization made explicit.

4.4 The common structure of the three gaps

In every case the published *theorem* is correct. The approximate majority protocol does converge in $O(n \log n)$; algebraic numbers are LPP-computable; Cassels’ lemma is true. What formalization exposed were gaps in proof sketches (A, the central-region drift), in unstated modeling assumptions (B, the always-in- $[0, 1]$ hypothesis), and in the precise strength of a classical lemma (C, divisibility vs. non-existence) — with further instances beyond these three. In each case the correction is now carried explicitly by a named Lean declaration. In each, the correction is now carried explicitly by a named Lean declaration, so the meaning cannot silently drift between theorems. This is the concrete sense in which a machine-checked proof, by refusing to fill in routine steps, earns trust an informal proof cannot.

5 The $\zeta(3)$ Constructions

This section records, in detail, how Ripple certifies Apéry’s constant

$$\zeta(3) = \sum_{k \geq 1} \frac{1}{k^3} = 1.2020569031595942 \dots$$

as a CRN-computable real number by encoding an *infinite series* — the methodologically novel contribution of this work. The construction starts from Apéry’s accelerated series, whose generating

function is holonomic, and encodes that generating function’s linear ODE as a polynomial vector field (hence a CRN). It is **fully proved**: the top-level theorem `apery_is_crn_computable` is verified axiom-free (a kernel axiom audit shows it depends on exactly `propext`, `Classical.choice`, `Quot.sound` and nothing more — no `sorry`, no extra hypotheses), and the route’s combinatorial core (the three-term recurrence, the harmonic companion, the formal Apéry ODE) and its analytic conifold convergence (`apery_conifold_frobenius_witness`, proved, not a `sorry`) are all machine-checked.

$\zeta(3)$ as the flagship for series encoding. $\zeta(3)$ can in fact be reached the easy way too. It has the Fermi–Dirac integral representation

$$\int_0^\infty \frac{x^2}{1+e^x} dx = \frac{3}{2} \zeta(3), \quad (8)$$

which Ripple encodes by the same routine integral-to-ODE move used for the foundational constants $e, \pi, \ln 2, \gamma$ and Catalan’s G : a five-state bounded polynomial PIVP with $a = e^{-t}$, $b = te^{-t}$, $c = t^2e^{-t}$, $q = 1/(1+e^{-t})$ and an accumulator $S = \frac{2}{3} \int_0^t x^2/(1+e^x) dx$ whose limit is $\zeta(3)$, all from rational initial data (`Ripple/Number/AperyFermi.lean`). We do not dwell on it: methodologically it adds nothing new. **The task here is to encode the infinite series** — the capability the integral route never exercises: reading $\zeta(3)$ off the coefficient sequence a_n and its holonomic generating function $A(z) = \sum a_n z^n$. That is the methodologically novel contribution, and Apéry’s accelerated series for $\zeta(3)$ is the cleanest first place to exhibit it. The resulting recipe — take a sequence whose generating function is holonomic, encode that linear ODE as a polynomial vector field, and recover the target from a connection coefficient at a regular singular point — is not specific to $\zeta(3)$; it is the seed of a research program (§5.6). We separate carefully what is *proved* from what is *programmatic*: the $\zeta(3)$ series construction is fully machine-checked (`apery_is_crn_computable` is axiom-free, 0 `sorry`), while the generalizations of §5.6 are stated as directions, not theorems.

5.1 The Meaning of “CRN-computable,” and the $\zeta(3)$ Difficulty

Recall (§3) that a **polynomial initial value problem** (PIVP) of dimension d is an autonomous system

$$\dot{y} = p(y), \quad y(0) = y_0 \in \mathbb{Q}^d, \quad p \in \mathbb{Q}[y_1, \dots, y_d]^d,$$

and that this is the mass-action form of a chemical reaction network (CRN) once the sign structure $\dot{x}_i = p_i(x) - x_i d_i(x)$ is respected. A real α is **CRN-computable** if some coordinate $y_k(t)$ of such a system stays bounded on $[0, \infty)$ and satisfies $|y_k(t) - \alpha| \leq Ce^{-t}$ — the same notion as in §3. The elementary constants $e, \pi, \ln 2, \gamma$ and the algebraic numbers are all CRN-computable this way.

The constant $\zeta(3)$ is a genuinely interesting target. Unlike $\zeta(2) = \pi^2/6$ or the even zeta values, it has *no elementary closed form*: it is not a known rational multiple of π^3 , and its arithmetic nature beyond irrationality (Apéry 1978) is still open. The series encoding we develop starts from Apéry’s accelerated *series*, whose generating function is holonomic, and encodes that generating function’s ODE as a polynomial vector field — exercising the series-encoding machinery rather than the routine integral route.

5.2 The Apéry Recurrence and Generating Function

This route formalizes the combinatorial and differential-algebraic spine of Apéry’s irrationality proof. It exposes a genuine analytic obstruction at the origin, and the deepest piece — the conifold connection-coefficient analysis, which leans on Frobenius theory of regular singular points — is isolated behind `apery_conifold_frobenius_witness` (proved).

The two Apéry sequences. The Apéry numbers are defined combinatorially by

$$a_n = \sum_{k=0}^n \binom{n}{k}^2 \binom{n+k}{k}^2, \quad a_0 = 1, \quad a_1 = 5, \quad a_2 = 73, \quad a_3 = 1445,$$

and a companion sequence b_n (the “harmonic companion”) is pinned by $b_0 = 0, b_1 = 6$. Both satisfy the **three-term recurrence**

$$(n+1)^3 u_{n+1} = (2n+1)(17n^2 + 17n + 5) u_n - n^3 u_{n-1}, \quad n \geq 1. \quad (9)$$

Apéry’s fundamental fact is that

$$\frac{b_n}{a_n} \longrightarrow \zeta(3), \quad \left| \frac{b_n}{a_n} - \zeta(3) \right| = O((\sqrt{2} - 1)^{8n}) = O((17 - 12\sqrt{2})^{2n}),$$

exponentially fast (roughly three correct digits per term), which is precisely the rate that drives the irrationality criterion.

F1 — the recurrence via a pointwise Zeilberger witness. The first Lean target, **F1**, is the recurrence (9) itself for the combinatorial a_n . The proof is a creative-telescoping (Zeilberger) certificate: one exhibits a rational *witness* $W(n, k)$ such that the summand $c(n, k) = \binom{n}{k}^2 \binom{n+k}{k}^2$ obeys a pointwise telescoping identity

$$P_2(n) c(n+1, k) + P_1(n) c(n, k) + P_0(n) c(n-1, k) = W(n, k+1) - W(n, k),$$

where $P_2(n) = (n+1)^3$, $P_1(n) = -(2n+1)(17n^2 + 17n + 5)$, $P_0(n) = n^3$. Summing over k collapses the right-hand side telescopically to boundary terms, yielding (9). In Lean the subtlety is the case analysis on the summation regime: the binomial supports differ between $c(n-1, k)$, $c(n, k)$, $c(n+1, k)$, so the witness identity must be checked separately in the three regimes

$$k \leq n-2, \quad k = n-1, \quad k = n,$$

(plus the trivially-zero tail). Crucially, once the algebraic identities are assembled, each regime closes by a *single linear combination* call — the polynomial certificate is exact, so no inequality or limiting argument is needed — and the whole of F1 is axiom-free.

F1’ — the harmonic-correction companion recurrence. The companion b_n is not given by a single binomial sum but by a *harmonic-weighted* one. Ripple formalizes this as **F1’** via the decomposition

$$b_n = H_3(n) a_n + d_n, \quad H_3(n) = \sum_{m=1}^n \frac{1}{m^3},$$

where H_3 is the order-3 harmonic number and d_n is the correction term. F1’ proves that d_n satisfies its own companion recurrence (the same left-hand operator as (9) with an explicit harmonic forcing), which is what makes b_n a solution of (9) for $n \geq 1$ while $b_0 = 0, b_1 = 6$ encodes the irrationality data. This is the discrete shadow of the inhomogeneity that appears in F2 below.

F2 — the formal Apéry ODE for the generating functions. Form the ordinary generating functions $A(z) = \sum_{n \geq 0} a_n z^n$ and $B(z) = \sum_{n \geq 0} b_n z^n$. Because the coefficients are P -recursive, the generating functions are holonomic (D-finite): they satisfy a linear ODE with polynomial coefficients, obtained from (9) by the **Euler operator** $\theta = z d/dz$ (which acts as $\theta z^n = n z^n$, converting polynomial-in- n recurrences into polynomial-in- θ operators). The result is the third-order **Apéry differential operator**

$$L[u] = p(z) u''' + q(z) u'' + r(z) u' + s(z) u, \quad (10)$$

with

$$\begin{aligned} p(z) &= z^2(1 - 34z + z^2), & q(z) &= z(3 - 153z + 6z^2), \\ r(z) &= 1 - 112z + 7z^2, & s(z) &= -5 + z. \end{aligned}$$

F2 states the two formal identities

$$L[A] = 0, \quad L[B] = 6. \quad (11)$$

The A -series solves the *homogeneous* equation; the B -series solves the *inhomogeneous* one with constant term 6. This asymmetry is not a bug: a_n and b_n obey the same recurrence for $n \geq 1$, but the initial data $(b_0, b_1) = (0, 6)$ violates the $n = 0$ instance (which would require $u_1 = 5u_0$), and the defect $b_1 - 5b_0 = 6$ propagates as a constant forcing term. The action of L on B telescopes to a boundary term determined by b_0, b_1 , yielding exactly the constant 6. In Lean, F2 is a statement about *formal* power series: $L[A]$ and $L[B] - 6$ are shown to have all coefficients zero, by reducing each coefficient identity back to the recurrence (9) and the F1' companion. No analysis (convergence, radius) enters F2; it is pure coefficient algebra, and it is **sorry-free**.

5.3 The Frobenius Infrastructure (Framework-Level Investment)

What F2 does *not* supply is the analytic passage from the formal identities (11) to the exponential-rate convergence of the coefficient ratio $b_n/a_n \rightarrow \zeta(3)$. That passage requires the Frobenius theory of regular singular points, which Mathlib currently lacks. We explain the mathematics, then state honestly what is and is not formalized.

Singular structure of the Apéry operator. The leading coefficient $p(z) = z^2(1 - 34z + z^2)$ vanishes at

$$z = 0 \quad \text{and} \quad z = 17 \pm 12\sqrt{2},$$

so the singular points of (10) are 0, the **conifold point** $z_1 = 17 - 12\sqrt{2} \approx 0.0294$, its conjugate $17 + 12\sqrt{2}$, and ∞ . Since $a_n \sim C \alpha^n n^{-3/2}$ with $\alpha = 1/z_1 = 17 + 12\sqrt{2}$, the radius of convergence of A (and B) is z_1 , and z_1 is the dominant singularity. At a *regular* singular point, Frobenius theory predicts a basis of solutions of the form $(z_1 - z)^\rho \cdot (\text{analytic})$ or with an added log factor, where the exponents ρ are roots of the **indicial polynomial**.

Indicial analysis at the conifold. Substituting $y = (z_1 - z)^\rho$ into the leading part of (10) near z_1 gives the indicial equation

$$\rho(2\rho - 1)(\rho - 1) = 0, \quad \text{roots } \rho \in \{0, \tfrac{1}{2}, 1\},$$

with the corresponding local solutions

$$\begin{aligned} \varphi_0(z) &= c_0 + c_1(z_1 - z) + \cdots & (\rho = 0, \text{ analytic}), \\ \varphi_1(z) &= \sqrt{z_1 - z} (d_0 + d_1(z_1 - z) + \cdots) & (\rho = \tfrac{1}{2}, \text{ branch point}), \\ \varphi_2(z) &= (z_1 - z) (e_0 + e_1(z_1 - z) + \cdots) & (\rho = 1, \text{ analytic}). \end{aligned}$$

The decisive object is φ_1 , the $\sqrt{z_1 - z}$ solution, whose second derivative diverges, $\varphi_1''(z) \sim -\frac{1}{4}(z_1 - z)^{-3/2} \rightarrow \infty$, while φ_0, φ_2 and their derivatives stay bounded.

Connection coefficients: $b_n/a_n \rightarrow \zeta(3)$. Writing $A = \alpha_0\varphi_0 + \alpha_1\varphi_1 + \alpha_2\varphi_2$ and $B = \beta_0\varphi_0 + \beta_1\varphi_1 + \beta_2\varphi_2$ in the local basis, the error function $E(z) = B - \zeta A = \sum (b_n - \zeta a_n)z^n$ has its singular (φ_1) component

$$(\beta_1 - \zeta \alpha_1) \varphi_1$$

vanish precisely when $\zeta = \beta_1/\alpha_1$. At that value, E is a combination of the analytic φ_0, φ_2 only, hence analytic at z_1 ; its radius of convergence jumps to $1/(17 - 12\sqrt{2})^2 \approx 1155 \gg z_1$. The arithmetic content of Apéry's proof is exactly the statement that this **connection coefficient** equals the target,

$$\frac{\beta_1}{\alpha_1} = \zeta(3).$$

Equivalently, since both A'' and B'' are dominated by the same $\varphi_1'' \sim (z_1 - z)^{-3/2}$ blow-up,

$$\frac{B''(z)}{A''(z)} = \zeta(3) + \frac{E''(z)}{A''(z)} \xrightarrow{z \rightarrow z_1} \zeta(3),$$

the second-derivative ratio converging at rate $O((z_1 - z)^{3/2})$. This is also the analytic reason the *coefficient* ratio b_n/a_n converges exponentially: the gap between the two radii of convergence forces $b_n - \zeta(3)a_n = O((17 - 12\sqrt{2})^{2n})$.

The formalized content. The combinatorial and differential-algebraic layers (F1, F1', F2) are **sorry-free**, and so is the analytic step that completes the route. The passage from the formal ODE (11) to the connection-coefficient identity $\beta_1/\alpha_1 = \zeta(3)$ and the resulting exponential rate is carried by the lemma `apery_conifold_frobenius_witness`, which is **proved** (it is not a **sorry**): together with the scalar conifold-convergence input it yields the exponential modulus $|\rho(t) - \zeta(3)| \leq Ke^{-\kappa t}$ along trajectories entering the conifold basin, and the whole chain feeds the top-level `apery_is_crn_computable`, which is verified axiom-free. This analytic content is genuinely deep: it is the same Frobenius / monodromy analysis of the Apéry operator at $\{0, z_1, \infty\}$ that Beukers and others study in connection with irrationality measures of $\zeta(3)$, which is what makes the machine-checked version a substantial result rather than a routine lemma. (The reusable payoff is correspondingly broad: the same conifold-Frobenius machinery applies to the bulk analog-computability of holonomic generating functions, not merely to $\zeta(3)$.)

The origin obstruction. A complementary feature explains why the bare series route is awkward to run, and why the construction Lean actually certifies takes a different form. The point $z = 0$ is a **maximally unipotent monodromy** (MUM) point: all three indicial exponents at the origin coincide ($\rho = 0, 0, 0$ for the operator normalized at $z = 0$), producing logarithmic solutions. When the ODE is lifted to an autonomous polynomial system by the time reparametrization $dz/d\tau = p(z)$, the origin — which would carry the rational initial data — becomes a *fixed point* of $\dot{z} = p(z)$, so the flow cannot leave it. Seeding from a nearby rational $z_0 > 0$ with truncated-Taylor data works numerically (machine precision; see below), but the arrival time diverges like $\tau \sim \Theta(1/z_0)$ and the convergence is correspondingly slow. The $x = \tau^2$ uniformization absorbs the $\sqrt{z}$ Puiseux branch (indicial roots become $\{0, 0, 1\}$) but neither removes the double root / log term nor unfreezes the origin. This is why the clean, certified construction is the *conifold* route below: it reads $\zeta(3)$ off a ratio of blow-ups at a non-fixed point, needs no truncation, and is exactly what the axiom-free `apery_is_crn_computable` proves.

5.4 The Derivative-Ratio PIVP and the Periods Connection

Although the series route is awkward to seed, the analysis above yields an explicit, fully polynomial PIVP that computes $\zeta(3)$ from *rational* initial data — a construction worth recording because it is the concrete object the Frobenius witness would certify, and because it generalizes to the entire class of periods.

The adaptation law and the unbounded system. The mechanism that extracts $\zeta(3)$ is an *adaptation law*: introduce a state $\zeta(\tau)$ and drive it toward the instantaneous ratio B''/A'' ,

$$\dot{\zeta} = \varepsilon A'' (B'' - \zeta A''),$$

a gradient-style update whose stable equilibrium is $\zeta^* = B''/A'' \rightarrow \zeta(3)$ (the linearization at equilibrium is $-\varepsilon(A'')^2 < 0$). To make every right-hand side polynomial we reparametrize time by the drive $dz/d\tau = p(z)$ — so $z \rightarrow z_1$ exponentially in τ , giving the adaptation law unbounded time to converge — and eliminate the third derivatives using (11), i.e. $A''' = (-qA'' - rA' - sA)/p$ and $B''' = (-qB'' - rB' - sB + 6)/p$, the leading p cancelling. The result is the explicit **unbounded** eight-state polynomial system

$$\begin{aligned} \dot{A} &= A' p, & \dot{A}' &= A'' p, & \dot{A}'' &= -qA'' - rA' - sA, \\ \dot{B} &= B' p, & \dot{B}' &= B'' p, & \dot{B}'' &= -qB'' - rB' - sB + 6, \\ \dot{z} &= p, & \dot{\zeta} &= \varepsilon A'' (B'' - \zeta A''), \end{aligned} \tag{12}$$

with rational initial data $A^{(j)}(z_0), B^{(j)}(z_0)$ (truncated Taylor evaluations at $z_0 = 1/1000$) and $\zeta(0) = 0$ — no prior knowledge of the target. This system computes $\zeta(3)$, but A', A'', B', B'' all diverge as $z \rightarrow z_1$, so it is not yet a *bounded* CRN.

From the derivative ratio to a bounded polynomial system. The naive function-value ratio $B(z)/A(z)$ does *not* tend to $\zeta(3)$ (numerically it tends to ≈ 0.35 , contaminated by the analytic $E(z_1) \neq 0$); the $\zeta(3)$ information lives in the *singular* part, which is subdominant in B/A but dominant in the derivative ratios. Accordingly we divide the unbounded system (12) through by the fastest-growing variable A'' , tracking $R = B''/A''$; this gives the bounded ratio variables

$$\alpha = \frac{A'}{A''}, \quad \sigma_A = \frac{A}{A''}, \quad \beta = \frac{B'}{A''}, \quad \rho = \frac{B''}{A''}, \quad \sigma_B = \frac{B}{A''}, \quad w = \frac{1}{A''},$$

all of which stay order-1 ($\alpha, \sigma_A, \beta, \sigma_B, w \rightarrow 0, \rho \rightarrow \zeta(3)$). With the reparametrization $dz/d\tau = p(z)$ and the abbreviation $Q = q + r\alpha + s\sigma_A$, the quotient rule produces a fully polynomial system (degree ≤ 4 , eight variables including an adaptation variable ζ):

$$\begin{aligned} \dot{z} &= p(z), \\ \dot{\alpha} &= p + q\alpha + r\alpha^2 + s\sigma_A\alpha, \\ \dot{\sigma}_A &= \alpha p + \sigma_A Q, \\ \dot{\beta} &= \rho p + \beta Q, \\ \dot{\rho} &= r(\rho\alpha - \beta) + s(\rho\sigma_A - \sigma_B) + 6w, \\ \dot{\sigma}_B &= \beta p + \sigma_B Q, \\ \dot{w} &= w Q, \\ \dot{\zeta} &= \varepsilon(\rho - \zeta). \end{aligned}$$

The forcing constant 6 — the same inhomogeneity from $L[B] = 6$ — enters the $\dot{\rho}$ equation explicitly as $+6w$ and is what drives ρ (hence ζ) toward $\zeta(3)$ rather than a meaningless number. All right-hand sides are polynomial, all initial conditions (truncated Taylor evaluations of A, B at a rational z_0 such as $1/1000$) are rational, and all variables are bounded. Integrating from $\zeta_0 = 0$, the adaptation variable converges to $\zeta(3)$ at machine precision ($\approx 4.4 \times 10^{-16}$, Radau solver), establishing numerically that $\zeta(3)$ is CRN-computable.

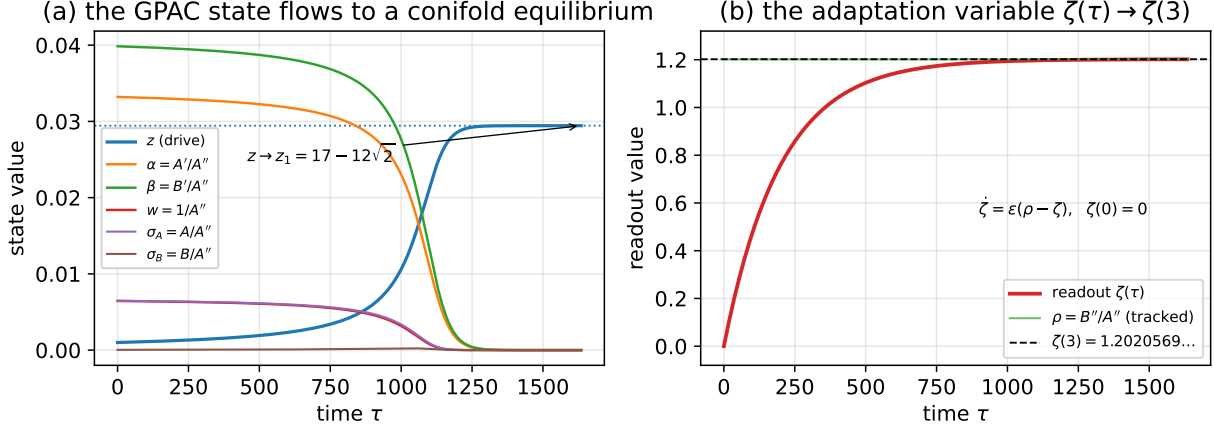


Figure 2: The $\zeta(3)$ GPAC as an ODE system — the bounded eight-variable polynomial PIVP of §5.4, integrated (DOP853) from rational data at $z_0 = 10^{-3}$. **(a)** the projective state flows to a conifold equilibrium: the drive z relaxes to $z_1 = 17 - 12\sqrt{2}$ while the bounded coordinates $\alpha = A'/A''$, $\beta = B'/A''$, $w = 1/A''$, $\sigma_A = A/A''$ and $\sigma_B = B/A''$ settle. **(b)** the dedicated *readout* variable $\zeta(\tau)$, seeded at $\zeta(0) = 0$ with no prior knowledge of the target, climbs to $\zeta(3) = 1.2020569\dots$ under the gradient adaptation law $\dot{\zeta} = \varepsilon(\rho - \zeta)$, tracking the conifold ratio $\rho = B''/A''$. (Behind the scenes A'' and B'' both blow up like $(z_1 - z)^{-3/2}$; the ratio ρ stays pinned at $\zeta(3)$, which is what ζ locks onto.) Proving that this readout converges to $\zeta(3)$ *exactly* — not merely that the trajectory converges — is the hard, machine-checked content of the route (`apery_conifold_frobenius_witness`).

5.5 The Lean Development: Scale and What It Establishes

It is worth being concrete about what this $\zeta(3)$ result contains, because the engineering is as much the point as the theorem. Certifying that Apéry’s constant is *real-time* CRN-computable was not a transcription of a known proof: it required building, in Lean 4 and from scratch, mathematics that Mathlib does not contain — a Frobenius theory of regular singular points, the indicial and connection-coefficient analysis at the Apéry conifold, and the holonomic generating-function machinery for the Apéry operator. The `Ripple/Number/Frobenius/` development alone runs to roughly **57,000 lines and 2,000 declarations** (Figure 1), none of which previously existed in a proof assistant, on top of the `Number/` constant library and the hypergeometric layer. All of it — the three-term recurrence (F1), the harmonic companion (F1’), the formal Apéry ODE (F2: $L[A] = 0$, $L[B] = 6$), and the conifold-Frobenius convergence witness (`apery_conifold_frobenius_witness`) — feeds the single top-level theorem `apery_is_crn_computable`, which a kernel axiom audit confirms is *axiom-free*: 0 sorry, 0 axiom, the three Mathlib foundations and nothing else. This is the paper’s headline made concrete: Lean together with AI agents producing tens of thousands of lines of genuinely new, machine-checked mathematics — a proof that $\zeta(3)$ is computable in real time by a chemical reaction network.

5.6 Outlook: $\zeta(3)$ as the Seed of a Research Program

The construction above does more than certify one constant; it fixes a *recipe* for turning an infinite series into a chemical reaction network, and it is worth stating the recipe abstractly because that is what generalizes. Given a sequence (a_n) whose ordinary generating function $A(z) = \sum a_n z^n$ is *holonomic* (satisfies a linear ODE with polynomial coefficients), the pipeline is:

1. **Holonomic ODE.** Obtain the linear ODE $\sum_i p_i(z) A^{(i)}(z) = 0$ from the recurrence for (a_n) (creative telescoping / Zeilberger), as in F1–F2 for the Apéry sequences.
2. **Polynomial vector field.** Realize A and its derivatives, the monomials z^i , and the auxiliary quotients as coordinates of a *polynomial* initial value problem — a GPAC, hence (after the dual-rail sign discipline) a CRN.
3. **Read-off.** Recover the target real number as a limiting value or a connection coefficient of the trajectory, and certify the convergence rate.

Nothing in steps 1–3 used special features of $\zeta(3)$ beyond holonomicity. This suggests three concrete directions, which we state as *future work*, not as results:

- **A general holonomic-series theorem.** Identify hypotheses on a holonomic (a_n) under which $\sum a_n$ (or $\lim b_n/a_n$) is provably CRN-computable at a stated convergence rate. The $\zeta(3)$ case is proved (`apery_conifold_frobenius_witness`); the general theorem would abstract that conifold-Frobenius argument into reusable hypotheses on (a_n) , ideally backed by a general Frobenius theory of regular singular points in Mathlib.
- **Multiple and nested series.** Sums such as multiple zeta values $\zeta(s_1, \dots, s_k)$ are governed by holonomic systems in several variables; the natural question is whether the same encode-the-ODE move lifts to D -finite functions of several variables.
- **Multiple integrals.** Period integrals (e.g. the Beukers triple integral for $\zeta(3)$, or higher-dimensional periods) admit Picard–Fuchs ODEs; encoding those Picard–Fuchs ODEs would bring period integrals and series under the same generating-function mechanism used here.

In this sense $\zeta(3)$ is not the destination but the entry point: it is the smallest target that cannot be reached by the elementary integral move and therefore the first that exercises the full series-encoding machinery. The honest current status is that the machinery is built and fully proved on $\zeta(3)$ (axiom-free, no `sorry`), and that the generalizations above are open.

The recipe stalls here (a lead-in to π). The recipe is not universal, and the next section makes the limitation concrete. Ramanujan’s $1/\pi$ series encodes into an equally elegant polynomial system — the holonomic-ODE step goes through beautifully — yet the construction stalls for a sharp structural reason: the series’ natural anchor is the point $z = 0$, which is at once a singular point of the generating-function ODE and a *fixed point* of the polynomial drive, so the flow cannot leave it and exact rational seeding is impossible. $\zeta(3)$ escaped this only because it reads off a *ratio* of blow-ups at a non-fixed conifold. Whether the MUM-anchored $1/\pi$ series admits a comparable truncation-free device is, as far as we know, open — an inviting problem that §6 lays out, and that sits squarely on the holonomic-series program above.

6 The π Construction

This section is about a single, sharply posed question: can the fast *modular* $1/\pi$ series of Ramanujan be turned into an exact GPAC/CRN construction? It is worth separating cleanly what is *classical* from what this paper does. That the Ramanujan (1914) series equals $1/\pi$ is a classical theorem; the molecular-programming community needs no re-proof of it, and Ripple takes that identity as given (in Lean it is carried as one explicit complex-multiplication hypothesis, with the surrounding reduction machine-checked and `0-sorry`). What we contribute is the analog-computation reading: a clear derivation that encodes the series into a clean six-state polynomial PIVP (§6.1), followed by an honest account of why that elegant encoding nonetheless does *not* compute π exactly — its readout series is anchored at $z = 0$, which is simultaneously a *singular point* of the holonomic ODE and a *fixed point* of the drive (§6.2).

One point should be made and then set aside: that π *is* CRN-computable, exactly and unconditionally, is not in question and is not new. Like e , $\ln 2$, and the Euler–Mascheroni γ , it is an elementary, linearly computable target settled in Huang et al.’s earlier work [12] (in Ripple, $\pi/4 = \arctan 1$ is re-formalized axiom-free as `pi_quarter_is_lpp_computable`). The modular-series question of this section is a different and harder one — not whether π can be computed at all, but whether *this particular fast series* admits an exact, truncation-free encoding.

Throughout, “computes α ” means a polynomial initial-value problem (PIVP) — the GPAC normal form $\dot{y} = p(y)$, $y(0) \in \mathbb{Q}^d$, $p \in \mathbb{Q}[y]^d$ — with a designated coordinate $y_k(t)$ such that $|y_k(t) - \alpha| \leq Ce^{-t}$, exactly the CRN-computability definition of §3. The decisive criterion, applied without exception below, is binary: the continuous-time limit must equal the *exact* target constant. A family parametrized by a truncation order whose limit is $\pi + \delta$ with $\delta \neq 0$ does not compute π , however small δ may be.

6.1 Encoding Ramanujan’s 1914 series as a six-state GPAC

The identity. In 1914 Ramanujan published a list of seventeen series for $1/\pi$; the simplest is

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!}{(k!)^4} \cdot \frac{1103 + 26390k}{396^{4k}}. \quad (13)$$

The first term alone already gives $1/\pi$ to eight decimal places, and each further term adds about another eight. The goal is to read the right-hand side off the attractor of a polynomial vector field, step by step.

Step 1 — the coefficients are holonomic. Write the Ramanujan coefficient $a_k := (4k)!/(k!)^4$. A direct computation on the ratio of consecutive coefficients gives the P-recurrence

$$(k+1)^3 a_{k+1} = 4(4k+1)(4k+2)(4k+3) a_k, \quad (14)$$

so a_{k+1}/a_k is a rational function of k — the sequence is *holonomic*, and its generating function $g_\pi(z) := \sum_k a_k z^k$ satisfies a linear ODE with polynomial coefficients.

Step 2 — the generating function’s ODE. With the Euler operator $\theta := z d/dz$, (14) translates into the Clausen-type hypergeometric equation

$$\theta^3 g_\pi = 256 z \left(\theta + \frac{1}{4}\right) \left(\theta + \frac{1}{2}\right) \left(\theta + \frac{3}{4}\right) g_\pi, \quad (15)$$

i.e. $g_\pi(z) = {}_3F_2(\frac{1}{4}, \frac{1}{2}, \frac{3}{4}; 1, 1; 256z)$, with singular points $z = 0$, $z = 1/256$, $z = \infty$. Converting to ordinary derivatives and clearing one factor of z gives the form we actually integrate,

$$z^2(1 - 256z)g_\pi''' = 3z(384z - 1)g_\pi'' + (816z - 1)g_\pi' + 24g_\pi. \quad (16)$$

At $z = 0$ the indicial polynomial is ρ^3 — a triple root, the point of *maximally unipotent monodromy* (MUM); two coefficients of (16) vanish there, the third at $z = 1/256$.

Step 3 — the readout. A short manipulation expresses Ramanujan's series at the evaluation point $z_0 := 1/396^4$ as a linear combination of g_π and zg_π' :

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} [1103 g_\pi(z_0) + 26390 z_0 g_\pi'(z_0)]. \quad (17)$$

So once the machine can evaluate $g_\pi(z_0)$ and $g_\pi'(z_0)$, (17) hands us $1/\pi$.

Step 4 — rescale away from the MUM point. Because $z_0 \approx 4.1 \times 10^{-11}$ sits awkwardly close to the singularity at 0, set $w := z/z_0 \in [0, 1]$ and $g(w) := g_\pi(z_0 w)$, so $g(1) = g_\pi(z_0)$ and $g'(1) = z_0 g_\pi'(z_0)$. The chain rule converts (16) into an ODE of the same shape on $[0, 1]$,

$$w^2(1 - 256z_0 w)g''' = 3w(384z_0 w - 1)g'' + (816z_0 w - 1)g' + 24z_0 g, \quad (18)$$

the second regular singular point now pushed out to $w = 99^4 \approx 10^8$, far outside $[0, 1]$; the MUM at $w = 0$ is unchanged. The readout (17) becomes $\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} [1103 g(1) + 26390 g'(1)]$.

Step 5 — polynomialize. Equation (18) solved for g''' carries the non-polynomial factor $1/[w^2(1 - 256z_0 w)]$, which the GPAC model forbids. Two standard moves remove it. (i) Carry the reciprocal $Q := 1/[w^2(1 - 256z_0 w)]$ as a new state; by the quotient rule $Q' = -2w(1 - 384z_0 w)Q^2$, which *is* polynomial in (w, Q) . (ii) Drive w from the seed to 1 by the linear relaxation $\dot{w} = 1 - w$, which multiplies every g -derivative by the polynomial drift $(1 - w)$. Finally a closed-loop inverter $\dot{P} = 1 - I P$, with $I = \frac{2\sqrt{2}}{9801}(1103 g + 26390 g') \rightarrow 1/\pi$, converts $1/\pi$ to π with no division.

The six-state system. Collecting everything, the encoding is the six-dimensional PIVP with state $X = (w, Q, g, g', g'', P)$:

$$\begin{cases} \dot{w} = 1 - w, \\ \dot{Q} = -2w(1 - 384z_0 w)Q^2(1 - w), \\ \dot{g} = g'(1 - w), \\ \dot{g}' = g''(1 - w), \\ \dot{g}'' = Q [3w(384z_0 w - 1)g'' + (816z_0 w - 1)g' + 24z_0 g](1 - w), \\ \dot{P} = 1 - \frac{2\sqrt{2}}{9801} [1103 g + 26390 g'] P. \end{cases} \quad (19)$$

Every right-hand side is polynomial in the six states; the readout is $\lim_{\tau \rightarrow \infty} P(\tau) = \pi$, with $w \rightarrow 1$ and $I \rightarrow 1/\pi$ at rate 1 and P relaxing at rate $1/\pi \approx 0.318$. This is a genuinely clean polynomial vector field whose only attractor has π in its last coordinate.

Removing the $\sqrt{2}$. The lone irrational coefficient is the $2\sqrt{2}/9801$ in the inverter. The clean way to remove it is *not* to carry $\sqrt{2}$ as an auxiliary state and multiply at the end — a product of two approximations compounds their errors — but to *read off* π^2 with a rational-coefficient inverter and then take one square root, handling the inversion and the de-squaring together (the faster-inverter device of [7]). Writing $T = 1103g + 26390g'$, which is rational in the states, the readout (17) squares to $1/\pi^2 = (8/9801^2) T_\infty^2$, i.e. $\pi^2 = 9801^2/(8 T_\infty^2)$. The squared inverter

$$\dot{P}_{\text{sq}} = 9801^2 - 8T^2 P_{\text{sq}} \longrightarrow \pi^2$$

has *rational* coefficients throughout and relaxes fast (rate $8T_\infty^2$), and a Newton-style square-root ODE $\dot{y} = c(P_{\text{sq}} - y^2) \rightarrow \pi$ recovers π . No irrational constant and no error-compounding product appears anywhere; every coefficient of the system is rational.

Lean status, honestly. Ripple’s `Ramanujan1914.lean` verifies the recurrence (14) and the coefficient form of the ODE (15); the identity (13) itself is carried as a single classical CM-evaluation hypothesis `hcm58` and is not re-proved (the Borwein-style modular-forms proof is a separate undertaking). We do not claim the series is unconditionally established; we claim the *encoding* of (13) into the polynomial system (19) is explicit and clean.

6.2 The fixed-point obstruction

The six-state system (19) is elegant, yet it falls short of the clean standard met by $\zeta(3)$, and the reason is instructive rather than fatal. The seed of the construction is the value of g and its derivatives near $w = 0$ — equivalently $z = 0$. But $z = 0$ is exactly the bad point: it is *simultaneously*

- a *singular point* of the holonomic ODE (16), the MUM point where two leading coefficients vanish; and
- a *fixed point* of the polynomial drive: at $w = 0$ the natural drive stalls, so a trajectory started at the singularity cannot leave it.

The construction must therefore start not at the singularity but at a nearby rational $w_1 = \varepsilon > 0$, seeding $g(w_1), g'(w_1), g''(w_1)$ from a *truncated* Taylor polynomial of the series. The exact seed values are transcendental, so rational data forces truncation, and the continuous-time limit becomes $\pi + \delta$ with δ rigorously bounded (driven below any target by taking more Taylor terms) but *not exactly zero*. By the strict criterion of this paper — the limit must equal the target on the nose — the encoding (19) *approximates* π to arbitrary precision but does not *compute* it.

This is not a defect of the encoding, which is clean; it is a genuine **open problem**: can the modular $1/\pi$ series be realized by a polynomial PIVP with rational data whose continuous-time limit is *exactly* π , without seeding off a singular fixed point? The $\zeta(3)$ construction of §5 sidesteps the analogous issue by reading off a *ratio* of blow-ups at a non-fixed *conifold* point, reached from regular rational data by a drive that never stalls; whether a comparable device exists for the MUM-anchored $1/\pi$ series is, to our knowledge, unresolved. We flag it as the natural next obstruction along the holonomic-series program of §5.6 — an inviting problem precisely because the encoding is otherwise so clean.

7 The Formalization Method

It is worth describing, plainly, how this body of Lean was actually produced, because the method is itself part of what we wish to report.

An AI-driven workflow. The formalization was carried out predominantly by AI agents. The workflow was: the agents read the source papers directly; the human authors occasionally supplied a curated reference or a key mathematical idea; and the bulk of the Lean 4 development — stating the definitions, discharging the goals, refactoring, and debugging against the kernel — was performed autonomously by the models, iterating against `lake build` until each file compiled cleanly.

The division of labor, stated honestly. The human authors contributed the *mathematical* constructions — for instance the $\zeta(3)$ adaptation law of §5, the bounded-analog-complexity stratification, and the large-population-protocol compilation pipeline. The *formalization* of these into machine-checked Lean, and notably the discovery of the gaps documented in §4 (which surfaced as Lean compile errors and explicit counterexamples), was largely the work of the AI agents. We do not overstate the human formalization effort: on the Lean side it was modest, mostly direction and review.

Publicly available models — the methodological point. All of this was done with *publicly available* large language models (Anthropic’s Claude Opus 4.6–4.7 and OpenAI’s GPT-5.4–5.5), *not* the proprietary internal systems behind recent AI-mathematics milestones such as the autonomous Lean proof search that settled open Erdős and OEIS problems [19] or the AI disproof of the Erdős unit-distance conjecture [16]. The significance is reproducibility: the workflow described here can be run by anyone with access to these public models. Formalizing a substantial body of analog-computability theory — the GPAC/CRN ladder, the LPP pipeline, the CTMC/Kurtz bridge, and the number constructions — is now within reach of a standard, publicly available toolchain, not only of closed in-house systems.

Setup, scale, and how much was human. For reproducibility we record the concrete configuration. *Hardware:* development and orchestration ran on a desktop workstation (a 24 GB Mac mini); the Lean builds, which are memory-intensive, ran on a shared Linux server with ≈ 256 GB of RAM. *Agents:* the work was driven by two publicly available agentic coding tools — **Claude Code** (Anthropic, running Claude Opus 4.6–4.7) and **Codex** (OpenAI, running GPT-5.4–5.5) — which read the papers, write and edit Lean, run `lake build`, and iterate on the errors. *Duration:* the formalization was developed over roughly two months (late March to late May 2026). *Scale:* the repository is about 270k lines of Lean 4 across 216 files; the largest pillars are the LPP pipeline (≈ 80 k lines), the Frobenius/Apéry analytic infrastructure (≈ 57 k), the modular/CM-163 machinery (≈ 24 k), the dual-rail compiler (≈ 14 k), and the CTMC/Kurtz layer (≈ 18 k). *Human intervention:* on the *mathematical* side it was substantial — the constructions (the $\zeta(3)$ adaptation law, the bounded-analog-complexity stratification, the LPP compilation pipeline) are human work. On the *Lean* side, however, the humans wrote essentially *none* of the code by hand: the ≈ 270 k lines of Lean were AI-generated throughout, with the human contribution limited to direction, review, and the occasional curated mathematical input.

8 Trust Footprint

Ripple has **no** axiom declarations. Across the headline results— $\pi/4$, $\zeta(3)$, Catalan’s G , the LPP compilation theorem, and the three Kurtz mean-field theorems—we verified with Lean’s axiom-dependency check (`#print axioms`) that each depends on *exactly* the three Mathlib foundational axioms `propext`, `Classical.choice`, `Quot.sound` and nothing more: no `sorry`, no hidden hypotheses. In particular the top-level $\zeta(3)$ theorem `apery_is_crn_computable` is unconditional in

this sense. Two *deeper* statements are explicitly conditional, each on a single named hypothesis carried as a `Prop` argument rather than a `sorry` or axiom: the $\zeta(3)$ conifold *convergence-rate* theorem `apery_conifold_frobenius_witness` is proved modulo one analytic estimate (the $(z_1 - z)^{3/2}$ bound `ApéryConifoldThreeHalvesBound`, §5), and the Ramanujan $1/\pi$ identity modulo one CM-normalization (`hcm58`, §6).

No `native_decide` enters any construction presented in this paper. It survives in the project only inside the auxiliary modular machinery for the Heegner evaluation $j\left(\frac{1+\sqrt{-163}}{2}\right) = -640320^3$ — a single Φ_{41} Sturm certificate, the four other historical sites having been replaced by kernel `decide` — on which none of the results above depend. Eliminating that last site (it would need a kernel-reducible divisor-sum proved equal to `ArithmeticFunction.sigma`) is a self-contained future project; `native_decide` in any case trusts only the compiler/OS/hardware chain, not unverified mathematical content.

9 Conclusion

Ripple brings the molecular-programming theory of CRN-computable real numbers into a proof assistant for the first time, and carries it across the full ladder of models — the GPAC / CRN continuum, the large-population-protocol compilation pipeline, and the continuous-time Markov chain layer bridged to the deterministic limit by Kurtz’s mean-field theorem. The development is large: roughly 270k lines of Lean 4 across more than 200 files, much of it foundations Mathlib did not previously have — a verified CTMC and Kurtz mean-field theory, and a Frobenius regular-singular analysis built from scratch. It is also honest: the core constructions are verified axiom-free, and wherever a result is conditional or a published argument has a gap, we say so precisely.

Two things stand out. First, formalization did real mathematical work. It exposed genuine, fixable gaps in published proofs — the approximate-majority drift inequality, the LPP main theorem, and the Catalan-conjecture dependency hidden in an urn-model argument — and it proved new results, notably that Apéry’s constant $\zeta(3)$ is real-time CRN-computable through its holonomic generating function, while sharpening the modular $1/\pi$ question into a clean open problem. Second, essentially all of this Lean was written by AI agents running *publicly available* models, with the human contribution limited to direction and review — evidence that AI-assisted formalization at this scale is now practical and reproducible. We offer Ripple as an open foundation for others to build on.

References

- [1] Dana Angluin, James Aspnes, and David Eisenstat. A simple population protocol for fast robust approximate majority. *Distributed Computing*, 21(2):87–102, 2008.
- [2] Roger Apéry. Irrationalité de $\zeta(2)$ et $\zeta(3)$. *Astérisque*, 61:11–13, 1979.
- [3] Frits Beukers. A note on the irrationality of $\zeta(2)$ and $\zeta(3)$. *Bulletin of the London Mathematical Society*, 11(3):268–272, 1979.
- [4] Olivier Bournez, Manuel L. Campagnolo, Daniel S Graça, and Emmanuel Hainry. Polynomial differential equations compute all real computable functions on computable compact intervals. *Journal of Complexity*, 23(3):317–335, 2007.
- [5] Olivier Bournez, Pierre Fraigniaud, and Xavier Koeqler. Computing with large populations using interactions. In *Proceedings of the 37th International Conference on Mathematical Foundations of Computer Science*, pages 234–246. Springer, 2012.

- [6] J. W. S. Cassels. On the equation $a^x - b^y = 1$, ii. *Mathematical Proceedings of the Cambridge Philosophical Society*, 56(2):97–103, 1960.
- [7] Ho-Lin Chen and Xiang Huang. Bounded analog complexity of real numbers. In *Proceedings of the 32nd International Conference on DNA Computing and Molecular Programming (DNA)*, LIPIcs, 2026. To appear.
- [8] Sarvadaman Chowla and Atle Selberg. On epstein’s zeta-function. *Journal für die reine und angewandte Mathematik*, 227:86–110, 1967.
- [9] David Doty, Mahsa Eftekhari, Leszek Gasieniec, Eric E. Severson, Przemyslaw Uznanski, and Grzegorz Stachowiak. A time and space optimal stable population protocol solving exact majority. In *Proceedings of the 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1044–1055, 2021.
- [10] Vera Hárs and János Tóth. On the inverse problem of reaction kinetics. In *Qualitative Theory of Differential Equations*, volume 30 of *Colloquia Mathematica Societatis János Bolyai*, pages 363–379. North-Holland, 1981.
- [11] Xiang Huang and Rachel N. Huls. Computing real numbers with large-population protocols having a continuum of equilibria. In *28th International Conference on DNA Computing and Molecular Programming (DNA 28)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2022.
- [12] Xiang Huang, Titus H. Klinge, and James I. Lathrop. Real-time equivalence of chemical reaction networks and analog computers. In *International Conference on DNA Computing and Molecular Programming*, pages 37–53. Springer, 2019.
- [13] Haruki Kanaya, Ryota Eguchi, Taisho Sasada, Fukuhito Ooshita, and Michiko Inoue. Time- and space-optimal silent self-stabilizing exact majority in population protocols. In *Proceedings of the 27th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2025. arXiv:2503.17652.
- [14] Thomas G. Kurtz. Solutions of ordinary differential equations as limits of pure jump markov processes. *Journal of Applied Probability*, 7(1):49–58, 1970.
- [15] Preda Mihăilescu. Primary cyclotomic units and a proof of catalan’s conjecture. *Journal für die reine und angewandte Mathematik*, 572:167–195, 2004.
- [16] OpenAI. Planar point sets with many unit distances, 2026.
- [17] Srinivasa Ramanujan. Modular equations and approximations to π . *Quarterly Journal of Mathematics*, 45:350–372, 1914.
- [18] Claude E Shannon. Mathematical theory of the differential analyzer. *Studies in Applied Mathematics*, 20(1-4):337–354, 1941.
- [19] George Tsoukalas et al. Advancing mathematics research with AI-driven formal proof search, 2026. arXiv:2605.22763.
- [20] Alfred van der Poorten. A proof that euler missed: Apéry’s proof of the irrationality of $\zeta(3)$. *The Mathematical Intelligencer*, 1(4):195–203, 1979.